

CHAPTER 6 — BINARY SEARCH BASICS

Learning Goal

Understand how **Binary Search works by repeatedly dividing the search area into two parts**, and why the array needs to be **sorted**.

SLIDE 1 — Meet Binary Search

A Faster Way to Search

Imagine you're looking for a name in a **sorted list** .

Instead of checking every name from the beginning, you can:

- Go to the **middle**
- Compare with your target
- Decide whether to search **left or right**

That's the basic idea of **Binary Search**.

Example

Sorted array:

10 20 30 40 50 60 70
 ↑
 Middle

 Target = 60

60 > 40

So we can **ignore everything on the left** 

10 20 30 40  | 50 60 70
 ↑
 Search here

Memory Trick

BINARY = DIVIDE INTO TWO

SLIDE 2 — Why Must the Array Be Sorted?

Binary Search Needs Order!

Binary Search decides where to continue searching based on the comparison with the middle value.

So the values must be arranged in **ascending or descending order**.

Sorted

10 → 20 → 30 → 40 → 50 → 60

 Binary Search can work.

Unsorted

40 → 10 → 60 → 20 → 50 → 30

 Binary Search cannot safely decide which half to ignore.

Think Like This:

If:

 Target 60

Middle = 40

Because:

60 > 40

👉 In a **sorted ascending array**, target must be on the **right side**.

💡 **Memory Trick**

Sorted → Decide → Discard Half

🟪 SLIDE 3 — The Power of the Middle Element

🎯 **Why Do We Check the Middle?**

The **middle element** helps us decide which half contains the target.

Example:

[10] [20] [30] [40] [50] [60] [70]
 ↑
 Middle

🎯 Target = 20

Compare:

20 < 40

👉 Search the **left half**.

[10] [20] [30] | [40] [50] [60] [70]
🔍 Search ❌ Ignore

Another example:

🎯 Target = 60

60 > 40

👉 Search the **right half**.

[10] [20] [30] | [40] [50] [60] [70]
❌ Ignore 🔍 Search

Three Decisions

$\text{Target} == \text{Middle} \rightarrow$  Found

$\text{Target} < \text{Middle} \rightarrow$  Left half

$\text{Target} > \text{Middle} \rightarrow$  Right half

Memory Trick

Compare with Middle $\rightarrow$ Choose Left or Right

SLIDE 4 — Binary Search Keeps Shrinking the Search Area

 Divide $\rightarrow$ Ignore $\rightarrow$ Search Again

Suppose:

[10] [20] [30] [40] [50] [60] [70] [80]

 Target = 70

Step 1

Middle = 40

70 > 40

 Ignore left half.

10 20 30 40 | 50 60 70 80



Step 2

Check the middle of the remaining area.

70 > 60

✗ Ignore the smaller left part again.

50 60 | 70 80



Step 3

🎯 70 → ✅ Found!

🚀 Why is this useful?

Instead of checking every value, Binary Search **keeps reducing the search area**.

💡 Memory Trick

Middle → Compare → Remove Half → Repeat

🟪 SLIDE 5 — Linear Search vs Binary Search

🔪 Two Ways to Search

Feature	🔍 Linear Search	⚡ Binary Search
Main idea	Check one by one	Divide into halves
Data	Can be unsorted	Must be sorted
Starting point	Beginning	Middle
Decision	Keep moving	Left or right
Search area	Reduces one element at a time	Reduces by half

📦 Example

Linear Search:

10 → 20 → 30 → 40 → 50 → ...

👣 One by one

Binary Search:

10 20 30 | 40 | 50 60 70

✂️ Divide and eliminate

🧠 BIG MEMORY TRICK

🔍 **LINEAR = ONE BY ONE**
⚡ **BINARY = HALF BY HALF**

🧠 CHAPTER 6 — QUIZ

● Q1 — Fill the Code

Complete the condition for checking whether the target is equal to the middle element:

```
if(target _____ numbers[mid])
{
    printf("Found");
}
```

💡 **Hint:** We need to check whether both values are equal.

📖 Glossary

- 🎯 **Target** — The value we want to find.
- 📦 **Middle** — The element at the center of the current search area.

✓ **Answer:** ==

● Q2 — Fill the Code

A sorted array is given:

```
int numbers[] = {10, 20, 30, 40, 50};  
int mid = 2;  
int target = 20;
```

```
if(target < numbers[mid])  
{  
    printf("Search the _____ half");  
}
```

💡 **Hint:** `numbers[2]` is 30, and `20 < 30`.

📖 Glossary

- 🎯 **Target** — The value being searched for.
- 📄 **mid** — The index representing the middle position.
- 📦 **Half** — One of the two parts of the search area.

✓ **Answer:** left

● Q3 — MCQ

Which condition is required for Binary Search?

- A) The array must contain only even numbers
- B) The array must be sorted
- C) The array must contain duplicate values
- D) The array must contain exactly 10 elements

💡 **Hint:** Binary Search decides which half to search using the middle value.

📖 Glossary

- ⚡ **Binary Search** — A searching method that repeatedly divides the search area into two parts.
- 📊 **Sorted** — Values arranged in an order.

✅ **Answer: B) The array must be sorted**

🟡 Q4 — MCQ

Consider the sorted array:

10 20 30 40 50 60 70

🎯 Target = 60

Middle value = 40

Which half should Binary Search continue with?

- A) Left half
- B) Right half
- C) Both halves
- D) Stop immediately

💡 **Hint:** Compare 60 with 40.

📖 Glossary

- 🎯 **Target** — The value we want to find.
- 📊 **Middle value** — The value at the center of the search area.
- ➡ **Right half** — The part containing values greater than the middle in ascending order.

✅ **Answer: B) Right half**

🟡 Q5 — MCQ

What is the main idea of Binary Search?

- A) Check every element from the beginning
- B) Randomly select elements
- C) Divide the search area and continue with the relevant half
- D) Sort the array after every comparison

 **Hint:** Think about what happens after checking the middle element.

Glossary

-  **Binary Search** — A method that searches by repeatedly dividing the search area.
-  **Divide** — Split something into parts.
-  **Search area** — The portion of the array currently being checked.

 **Answer: C) Divide the search area and continue with the relevant half**

CH 6 KEY IDEA

Linear Search:

 One → One → One → One

Binary Search:

 Whole → Half → Half → Half

 **Linear = One by One**

 **Binary = Half by Half**